Borrowing Strength Across AI Systems: A Hierarchical Bayesian Predictor for Benchmark-Level Cold Starts

Elias T.-H. Shieh
University of Illinois Urbana-Champaign
tshieh2@illinois.edu
ORCID 0009-0006-2120-8830

Abstract

The Predictive AI Evaluation Competition (PAIEC) asks for the probability that an AI system answers an item correctly on a benchmark that was held out from training, with 0 to 31 revealed labels per system–benchmark pair. We submit a small, transparent predictor: a logistic model with a benchmark intercept, a subject ability fitted offline on the AI Measurement Data Bank, a subject-by-benchmark deviation and an item effect, all with Gaussian priors whose scales are estimated on training benchmarks. At test time the predictor pools every revealed label on the same anonymous benchmark, from any subject, computes a MAP fit, and returns the posterior-mean probability under a Laplace approximation. It uses only the Python standard library, no API calls and no learned text model. In a local replica of the streaming protocol with benchmark-level cross-validation (497 pairs on 72 held-out training benchmarks), it reaches a Brier ALC of 0.2055, against 0.2500 for a constant 0.5 and 0.2492 for the official empirical-mean baseline (difference -0.0438 [-0.0510, -0.0361], 95% cluster-bootstrap interval). An ablation attributes the gain mainly to pooling labels across subjects (-0.0099 [-0.0133, -0.0063]); the offline subject ability adds no detectable gain (-0.0005 [-0.0017, $+0.0008$]). We report what the predictor cannot do: at budget 0 it is barely better than 0.5, because benchmark difficulty is not predicted from item content. Two exploratory experiments, not part of the submitted predictor, address this gap: a bag-of-words difficulty model does not transfer across benchmarks, whereas LLM-judged item difficulty lowers Brier ALC by 4.7% in a small probe on 8 benchmarks (-0.0094 [-0.0164, $+0.0003$]).

1 Introduction

A decision maker who wants to know whether an AI system will work on a new task usually has a handful of labelled outcomes, not a full evaluation. PAIEC [AIMS Lab, Stanford University, 2026] turns this into a prediction task. A *subject* is an AI system (model, harness, harness version and reasoning effort); an *item* is a question or task. The program receives the subject's and the item's attributes, plus the outcomes revealed so far, and returns $P(y = 1)$. Test items come from benchmarks that are not in the public training release, so the evaluation is a benchmark-level cold start. Each subject–benchmark pair is scored at label budgets $n \in \{0, 1, 3, 7, 15, 31\}$, and the score is the Brier ALC,

$$\text{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}, \tag{1}$$

averaged equally over pairs, where B_n is the Brier score at budget n.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Predictive AI Evaluation Competition (PAIEC).

Two features of the interface shape our approach. First, the revealed evidence passed to the predictor contains labels "across sampled subject–benchmark pairs" [AIMS Lab, Stanford University, 2026], so while predicting subject s on benchmark b we can also see other subjects' outcomes on b. Second, 50% of the score weight sits on budgets 1–7, where a pair has at most seven labels of its own. A predictor that uses only the target pair's labels, such as the official empirical-mean baseline [Truong et al., 2026b], is noisy there; the baseline's Brier score at budget 1 is worse than a constant (Table 1).

Contributions. (i) A hierarchical Bayesian predictor that pools evidence across subjects on the same held-out benchmark and integrates over its own uncertainty (§2). (ii) A local replica of the streaming protocol with benchmark-level cross-validation and a cluster bootstrap over benchmarks (§3). (iii) Ablations showing where the gain comes from and where it does not (§4). (iv) Two exploratory experiments on predicting item difficulty from item text, one negative and one preliminary positive (§5), and a list of limitations (§6).

Related work. Item response theory (IRT) has been used to estimate an LLM's benchmark accuracy from a small subset of items; tinyBenchmarks [Maia Polo et al., 2024], for example, shows that 100 curated examples can suffice on MMLU. That line of work assumes "access to correctness evaluations for a set of LLMs that have been previously evaluated on the entirety of the benchmark" [Maia Polo et al., 2024], which is what lets it fit item parameters. PAIEC removes this assumption: the test benchmark has no prior evaluations, so item parameters cannot be fitted offline. Our model keeps the IRT form but learns the benchmark intercept, and item effects where possible, from the few labels revealed during evaluation. We did not run a systematic literature review in the time available; this paragraph is not a survey.

2 Method

Model. For subject s and item i on a held-out benchmark b,

$$\eta_{si} = c_b + \lambda\,\theta_s + u_{sb} + v_i, \qquad P(y_{si} = 1) = \sigma(\eta_{si}), \tag{2}$$

with $c_b \sim \mathcal{N}(m_0, \sigma_c^2)$, $u_{sb} \sim \mathcal{N}(0, \sigma_u^2)$ and $v_i \sim \mathcal{N}(0, \sigma_v^2)$. θ_s is a subject ability fitted offline and λ a transfer slope (both below). u_{sb} is the part of a subject's performance on b that θ_s does not explain. v_i is identified only when item i has been revealed for some subject; otherwise it is integrated out.

Offline fit. From the AI Measurement Data Bank [Truong et al., 2026a] we keep benchmarks with binary, item-level responses and public release status, and subject–benchmark cells with at least 20 distinct items. Subjects are keyed by their visible attributes (normalised name, harness, harness version, reasoning effort). For each cell we compute the smoothed log-odds accuracy $a_{sb} = \operatorname{logit}\big((k + 0.5)/(n + 1)\big)$ and fit the two-way model $a_{sb} = c_b + \theta_s + e_{sb}$ by alternating ridge least squares (prior variance 1 on θ_s). Then:

- m_0 and σ_c are the mean and standard deviation of the fitted c_b.
- λ and σ_u come from leave-one-benchmark-out: for each benchmark we refit θ without it and regress the benchmark's centred a_{sb} on the centred held-out θ_s. λ is the slope; σ_u^2 is the residual variance minus the median binomial sampling variance of a_{sb}.
- σ_v^2 is the median, over benchmarks with at least 20 items, of the variance of item log-odds accuracy pooled over subjects, minus its sampling variance.

The fit used 77 benchmarks (those with at least three distinct subject keys) and gave $m_0 = 0.02$, $\sigma_c = 1.29$, $\lambda = 0.65$, $\sigma_u = 0.53$ and $\sigma_v = 1.27$, with abilities for 1,175 subject keys. A test subject is looked up from the most specific key to the bare model name; failing that, it gets its provider's mean ability, and otherwise $\theta_s = 0$ with the population variance of $\lambda\theta$.

Test-time update. For a target on benchmark b, we collect every revealed label whose item carries the same anonymous `benchmark_id`, whichever subject it belongs to. Subjects are identified by their full attribute dictionary and items by a hash of their content, features and interactors. We maximise the log posterior of $(c_b, \{u_{sb}\}, \{v_i\})$ in Eq. 2, holding $\lambda\theta_s$ fixed, by coordinate-wise Newton steps (25 sweeps at most, step size capped at 2). Each coordinate's curvature gives a diagonal Laplace variance. The prediction for a target is the posterior mean probability, approximated as

$$\hat{p} = \sigma\Big(\mu \big/ \sqrt{1 + \pi V/8}\Big)\,, \tag{3}$$

where μ is the MAP linear predictor and V is the sum of the variances of c_b, u_{sb} and v_i (the prior variance for a subject or item with no labels; we add the variance of $\lambda\theta_s$ when the subject has no labels). This shrinks predictions toward 0.5 when evidence is thin, which matters for the Brier score. At budget 0 the prediction is $\sigma\big((m_0 + \lambda\theta_s)/\sqrt{1 + \pi V_0/8}\big)$ with $V_0 = \sigma_c^2 + \sigma_u^2 + \sigma_v^2 + \mathrm{Var}(\lambda\theta_s)$. Outputs are clipped to $[0.01, 0.99]$.

Acquisition. We submit no `labeling.py`, so the platform reveals labels at random. Random acquisition keeps the revealed labels representative of the benchmark, which the intercept c_b relies on; we did not test an uncertainty-based policy.

Implementation. `model.py` has about 200 lines, uses only the Python standard library and reads one JSON file of 300 KB. Fits are cached by a hash of the evidence, so all targets on one benchmark at one budget share one fit. A synthetic stress test (30 pairs on six benchmarks, 990 targets, 930 labels at budget 31, eight threads) runs all six budgets in under one second on our development machine.

3 Experimental setup

Local replica of the evaluation. The hidden test benchmarks are private, so we evaluate on training benchmarks with a benchmark-level split. We shuffle the 72 benchmarks that have at least one subject with 80 or more distinct items (the platform's threshold for a pair) into five folds. For each fold, the offline prior is refitted without the fold's benchmarks, and each fold benchmark is scored as follows, mirroring the competition description [AIMS Lab, Stanford University, 2026]: up to eight subjects with ≥ 80 distinct items are sampled; each pair's items are split 50/50 into acquisition and evaluation pools; acquisition order is random and budgets are nested prefixes of it; the evidence at budget n contains the first n acquisition labels of *every* sampled pair on that benchmark; a revealed label is one Bernoulli draw with the item's recorded success rate; and the Brier score covers all recorded responses of up to 150 evaluation items per pair. This gives 497 pairs on 72 benchmarks.

Uncertainty. Pairs on the same benchmark share the benchmark intercept and the pooled evidence, so they are not independent. All intervals are 95% percentile intervals from a cluster bootstrap that resamples benchmarks (2,000 resamples), and differences are paired within pair.

Assumptions we could not check. We do not know how many subjects the platform samples per benchmark, whether the acquisition/evaluation split is drawn per pair or per benchmark, or how test subjects overlap with training subjects. We therefore report a second split mode (one shared split per benchmark, so item effects are never identified for evaluation items) and vary the prior scales (§4).

Baselines and ablations. *Constant 0.5*. *Empirical mean*: the official baseline [Truong et al., 2026b], the unsmoothed mean of the target pair's revealed labels, 0.5 without labels. *Without subject prior*: $\lambda\theta_s = 0$ for every subject. *Without pooling*: only the target pair's own labels are passed to the model.

4 Results

Main comparison. Table 1 and Figure 1 summarise the local replica. Our predictor improves on the constant by -0.0445 [-0.0557, -0.0346] Brier ALC and on the official empirical-mean baseline by -0.0438 [-0.0510, -0.0361]. It has the lower ALC on 76% of pairs against the constant and 80% against the empirical mean. Most of the gain is at budgets 1–7: at budget 1, where the empirical mean scores 0.367, our predictor scores 0.215. At budget 31 the two differ by about 0.005, as expected once a pair has enough labels of its own.

Where the gain comes from. Pooling labels across subjects accounts for -0.0099 [-0.0133, -0.0063] of ALC. Without pooling, the model still beats the empirical mean, because shrinkage and the probit correction keep single labels from producing extreme predictions. The offline subject ability adds no detectable gain: -0.0005 [-0.0017, $+0.0008$] with pooling and -0.0005 [-0.0019, $+0.0011$] without. This held although 80% of simulated pairs matched a fitted ability at some key

Table 1: Local replica, benchmark-level five-fold cross-validation (497 pairs, 72 held-out benchmarks, per-pair split). Brier ALC averaged over pairs with a 95% cluster-bootstrap interval, and the mean Brier score at each budget. Lower is better.

Predictor	ALC	95% CI	B_0	B_1	B_3	B_7	B_{15}	B_{31}
Constant 0.5	0.2500	[0.250, 0.250]	0.250	0.250	0.250	0.250	0.250	0.250
Empirical mean (official)	0.2492	[0.232, 0.265]	0.250	0.367	0.253	0.212	0.195	0.190
Ours (full)	0.2055	[0.194, 0.215]	0.249	0.215	0.205	0.199	0.191	0.185
w/o subject prior	0.2060	[0.195, 0.216]	0.250	0.216	0.205	0.199	0.192	0.185
w/o pooling	0.2153	[0.206, 0.224]	0.249	0.234	0.220	0.206	0.196	0.191
w/o both	0.2158	[0.207, 0.224]	0.250	0.235	0.220	0.207	0.196	0.191

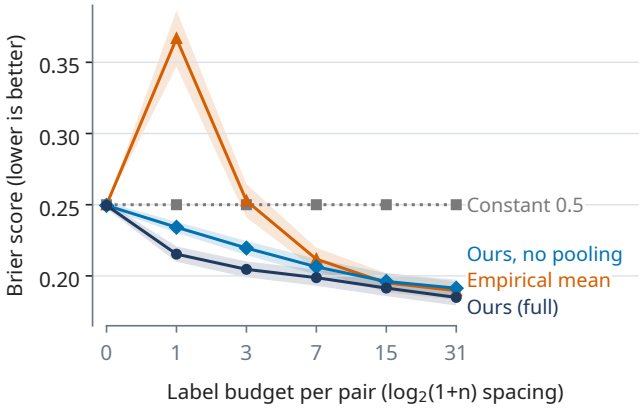


Figure 1: Mean Brier score per budget in the local replica; bands are ± 1.96 standard errors over pairs (not clustered, for display only). The empirical mean is worse than a constant at budget 1, where a single label sets the prediction to 0 or 1.

level (the rest fell back to the provider mean or the population). The fitted scales suggest why: the spread of $\lambda\theta_s$ (standard deviation 0.56 logits) is small next to the spread of benchmark intercepts (1.29) and of item effects (1.27), so knowing the subject moves predictions little, and once a few labels arrive u_{sb} absorbs it.

Item effects. With one shared split per benchmark, item effects are never identified for evaluation items; ALC is then 0.2049 and the pooling gain is -0.0089 [-0.0117, -0.0059], close to the per-pair split. With at most eight subjects and 31 labels each on benchmarks of hundreds of items, a revealed item rarely recurs among another subject's evaluation items, so the gain from pooling comes from learning c_b, not v_i.

Sensitivity to the prior scales. Table 2 halves and doubles each of σ_c, σ_u and σ_v with the others fixed. Halving σ_c or σ_u or doubling σ_v costs 0.002–0.006 ALC; the other three changes gain at most 0.001, less than the spread across seeds below. Over-confident priors on the benchmark intercept and the subject deviation hurt more than vague ones. These runs share the folds and seed of the main run, so they show sensitivity, not out-of-sample tuning; we did not change the submitted scales after seeing them.

Seeds. Rerunning the replica with different fold assignments, subject samples, splits and label draws gave ALC 0.2052–0.2064 over 3 seeds for the full predictor.

Platform result. At the time of writing, the Codabench formative evaluation of this submission had not returned a score; we therefore report only local results.

Table 2: Brier ALC of the full predictor when one prior scale is multiplied by 0.5 or 2 (local replica, same folds and seed as Table 1).

Scale	$\times 0.5$	$\times 1$ (submitted)	$\times 2$
σ_c	0.2098	0.2055	0.2047
σ_u	0.2076	0.2055	0.2046
σ_v	0.2049	0.2055	0.2110

Table 3: Exploratory LLM item-difficulty offset (not submitted). Local replica restricted to 640 judged items on 8 held-out benchmarks (51 pairs). Differences are paired, with 95% cluster-bootstrap intervals over benchmarks.

Predictor	ALC	Difference	B_0	B_3
Submitted (label-only)	0.2017	–	0.246	0.198
+ LLM offset, $\beta = 0.4$	0.1922	-0.0094 [-0.0164, $+0.0003$]	0.229	0.189
+ LLM offset, $\beta = 0.7$	0.1915	-0.0101 [-0.0233, $+0.0085$]	0.225	0.189
+ LLM offset, $\beta = 1.0$	0.1948	-0.0069 [-0.0261, $+0.0201$]	0.225	0.192

5 Exploratory: item difficulty from text (not in the submitted predictor)

Budget 0 carries 10% of the score, and item effects of evaluation items are never revealed, so information about an item before any label arrives would help at every budget. After submitting, we ran two experiments with the item text in measurement-db. Neither changes the submitted predictor; both enter Eq. 2 as a fixed item offset $\beta(\ell_i - \bar{\ell})$, where ℓ_i is a predicted log-odds of success for item i, and the offset also enters the fit of revealed labels.

Bag of words: negative. A ridge regression from hashed word uni- and bigrams and simple format statistics (length, digits, answer options, code) of the item text to the item's pooled log-odds accuracy, trained on 151 training benchmarks with benchmark-grouped five-fold cross-validation, did not transfer to held-out benchmarks: the best out-of-fold item-level R^2 was -0.02 and the within-benchmark correlation 0.11. The features mostly identify the benchmark. With out-of-fold predictions and $\beta = 0.5$ in the full replica, ALC changed by -0.0006 [-0.0013, $+0.0001$].

LLM judgments: preliminary positive. We asked an LLM (GPT-6-astra through the Codex command-line tool, 20 items per prompt) for the probability that "a typical AI system evaluated on this benchmark" answers each item, for 80 random items from each of 8 held-out benchmarks. The judged log-odds correlate with the items' pooled log-odds accuracy at 0.38 within a benchmark on average (from -0.22 to 0.63) and at 0.79 across the eight benchmark means. We reran the replica on these benchmarks, restricting evaluation items to the judged ones (51 pairs), with $\bar{\ell}$ the mean judged log-odds (Table 3). All three values of β lower ALC, mostly at budget 0, but every interval is wide, β was chosen on the same benchmarks, and one benchmark had a negative correlation. Asking for one item at a time, as a predictor would at scoring time, gave log-odds that correlate with the batch ones at 0.78 (80 items) but are more optimistic (mean 2.14 against 1.69), so the centring would have to be recalibrated. Using this at scoring time needs a paid API key and makes the predictor non-deterministic; a full test needs judgments for all evaluation items in the replica.

6 Limitations

- **Budget 0 is nearly uninformed.** B_0 is 0.249 against 0.250 for a constant. The predictor does not read item text, so it cannot tell a hard new benchmark from an easy one before labels arrive. An LLM-judged item offset looks promising in a small probe (§5) but was not submitted, because it needs an API key at scoring time and its evidence is still weak.
- **The local replica is our reading of the protocol.** The number of sampled subjects per benchmark, the split design and subject overlap between pools are unknown (§3). The training benchmarks may also differ from the private test pool. The local numbers rank methods under our assumptions; they do not predict the leaderboard score.

- **The subject prior failed to help** in our replica, and λ comes from leave-one-benchmark-out cells that are dominated by benchmarks with many subjects. Our subject keys rely on exact matches of normalised names.
- **Approximations.** The Laplace variances are diagonal and ignore the negative posterior correlation between c_b and u_{sb}, which overstates predictive variance somewhat. The hyper-parameters are point estimates; σ_v is estimated from item accuracy pooled over subjects, which mixes item difficulty with subject composition.
- **Scope of the evaluation.** One local seed was used for the ablations, sensitivity runs reuse the main run's folds, and the replica caps evaluation items at 150 per pair and subjects at eight per benchmark.
- **No acquisition policy.** We used random acquisition and did not test alternatives.

Data notes for the organisers. Checking the training release one level down (revision `54074453`), we recomputed the catalogue counts of the 160 benchmarks listed as binary, item-level and public. Response and subject counts match for all 160 and item counts for 159; for `ceval` the catalogue lists 13,951 items while the response table has 1,346 distinct item IDs. Four of these benchmarks (`chartmuseum`, `ineqmath`, `mmmu_dev_val`, `naturalreasoning`) contain only missing responses, and 46 others contain some missing responses; our pipeline drops missing values. The per-benchmark table is released with the code.

7 Reproducibility

Code, the fitted prior and the scripts for every number in this report are released at `https://github.com/ting-hong-shieh/paiec-hierarchical-bayes` (MIT licence). From the repository root:

1. download the binary item-level tables of measurement-db (revision `54074453`) with `tools/download_measurement_db.py`;
2. `python tools/build_tables.py -cache <cache> -out data/`;
3. `python tools/fit_prior.py -data data/ -out submission/prior.json`;
4. `python tools/simulate.py -data data/ -out results/sim_per_pair.json` (add `-split shared`, `-seed`, `-scale-sc` etc. for the other runs);
5. `python tools/analyze.py` to regenerate the tables and the figure;
6. zip `submission/` (model.py, prior.json, README.md) and check it with the official `check_submission_zip.py`.

Each simulation takes about three minutes on a 24-thread CPU and about 4 GB of memory. The exploratory runs use `tools/download_items.py`, `tools/text_prior.py`, `tools/llm_probe.py` and `simulate.py -model experiments/model_with_offset.py -text-beta` (with `-text-oof` or `-llm-probe`). The submitted predictor itself is deterministic and makes no network calls.

Disclosures

Training data. Only the AI Measurement Data Bank [Truong et al., 2026a], revision `54074453`, as listed in the competition's benchmark inventory: tables `benchmarks`, `subjects` and `responses` (and `items` for §5) of benchmarks with binary, item-level, public responses. The submitted predictor uses no item text, traces or embeddings; the exploratory experiments of §5 also use the item text in the `items` tables. **Pretrained models.** None in the submitted predictor; GPT-6-astra, through the Codex command-line tool, produced the item judgments of the exploratory probe. **Coding assistant.** The code and a first draft of this report were written with an LLM coding assistant (Claude Code, Anthropic) under the author's direction; the author checked the method, the numbers and the text. **Prior work reused.** The official baseline repository [Truong et al., 2026b] was used for its submission checker and as the empirical-mean comparison.

References

AIMS Lab, Stanford University. The predictive AI evaluation competition at NeurIPS 2026. https://aims.stanford.edu/competition, 2026. URL https://aims.stanford.edu/competition. Accessed 2026-11-07.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinybenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, 2024. URL https://arxiv.org/abs/2402.14992.

Nhi Truong, Sang T. Truong, and Sanmi Koyejo. The AI measurement data bank. https://huggingface.co/datasets/aims-foundations/measurement-db, 2026a. URL https://huggingface.co/datasets/aims-foundations/measurement-db. AIMS Lab, Stanford University. Revision 54074453.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, https://github.com/aims-foundations/paiec_baseline, 2026b. URL https://github.com/aims-foundations/paiec_baseline. Commit cc88a95.